AXION Agent Kit

Agent Kit — Detailed Spec (system-level)

Role

What it is: the packaged, self-contained bundle produced by ICP and consumed by KCP + build agents.
Purpose: give an agent everything it needs to build deterministically with enforceable constraints and auditable proof.

---

1) Agent Kit Goals (non-negotiable)

1. **Self-contained**: no missing context required to execute correctly
2. **Deterministic**: same kit + same policy → same decisions and comparable outputs
3. **Enforceable**: standards/gates/proof requirements are executable, not “suggestions”
4. **Auditable**: results and proofs can be ingested back into AXION without guesswork
5. **Least-privilege**: kit content is scoped to targets and executor type (internal/external)

---

2) Agent Kit Composition (what it contains)

2.1 Entry artifacts (always)

- **KIT_MANIFEST**
    - inventory of all included files/artifacts with hashes and schema versions
    - references pinset/lockfile

- **KIT_ENTRYPOINT**
    - reading order + “do this first” instructions
    - points to plan units and verification policy
- **PINSET / LOCKFILE**
    - exact versions used (templates/standards/gates/schemas/toolchain expectations)

2.2 Spec + constraint artifacts (included by profile)

- **CANONICAL_SPEC package**
    - canonical model needed to generate code and fill any remaining fields
- **STANDARDS_SNAPSHOT**
    - resolved standards and precedence decisions for this run
- **TEMPLATE_OUTPUTS**
    - render envelopes produced by ICP
    - any template selection result artifact
- **APPLICABILITY/SELECTION OUTPUTS** (optional but recommended)
    - why standards/templates were selected (audit)

2.3 Planning artifacts (when planning enabled)

- **PLAN_UNITS** (1 target per unit)
- **ACCEPTANCE_MAP** (unit → acceptance criteria → required proofs)
- **BUILD_PLAN / SEQUENCING** (deterministic order)

2.4 Execution + verification artifacts (always for serious builds)

- **VERIFICATION_POLICY**
    - required commands, configs, expected outputs, log rules
- **PROOF_TYPE SNAPSHOT** (if not embedded elsewhere)
    - allowed proof types + required fields
- **EVIDENCE POINTER RULES**
    - allowed pointer formats and resolver rules

2.5 Kit Control Plane (KCP) runtime (when shipped)

- KCP executable/CLI
- KCP configs (allowed commands, allowed paths, external restrictions)
- schema validators (as needed)

2.6 Safety + governance artifacts

- **REUSE_ALLOWLIST** (if reuse is permitted)
- **PLAGIARISM/IP POLICY** reference (or embedded rules)
- **SECRETS/PII HANDLING RULES** (especially for external kits)
- **AUDIT/OVERRIDE POLICY** subset (if KCP must enforce local override rules)

---

3) Agent Kit Variants (by executor type)

3.1 Internal Kit

- may include broader knowledge references
- may allow more tooling and wider repo access (still bounded)
- may include debug-heavy logs/configs

3.2 External Kit

- must exclude restricted knowledge items
- must include stricter guardrails (no copy rules, path allowlists)
- must minimize data exposure (only necessary specs/templates)
- must include explicit reuse_allowlist behavior

---

4) Agent Kit Directory Layout (logical)

This is the kit “shape” regardless of actual filenames:

- `kit/manifest/`
- `kit/entrypoint/`
- `kit/pins/`
- `kit/spec/`
- `kit/standards/`
- `kit/templates/`
- `kit/plan/`

- `kit/verify/`
- `kit/proof/`
- `kit/kcp/` (optional)
- `kit/policies/`
- `kit/logs/` (runtime output)
- `kit/results/` (RESULT files)
- `kit/reports/` (kit_run_report, validation report)

---

5) Kit Contracts (schemas you must lock)

5.1 KIT_MANIFEST schema must define

- kit_id, run_id
- schema_version(s)
- pinset reference/hash
- included artifact list:
    - artifact_id/type
    - path
    - hash
    - schema version
- required/optional classifications
- external/internal flags (if variant)

5.2 KIT_ENTRYPOINT schema must define

- reading order list (artifact pointers)
- execution steps list (commands/policies)
- unit execution rules (if planning included)
- verification invocation instructions
- result submission instructions (how to write RESULT files)

5.3 PLAN_UNIT schema must define (kit-local)

- unit_id
- targets[1]
- acceptance requirements
- required proofs
- output expectations
- pointers to relevant specs/templates

5.4 VERIFICATION_POLICY schema must define

- command list + required/optional flags
- expected outputs/log paths
- environment constraints
- failure and retry rules (if any)

5.5 RESULT schema must define

- unit_id
- implementation_refs[]
- proof_refs[]
- timestamps
- notes (optional)

---

6) Kit Execution Contract (what must happen)

When a kit is executed:

1. KCP validates kit integrity
2. KCP selects next unit deterministically
3. Build agent implements unit target
4. KCP runs verification per policy
5. KCP produces RESULT + proof objects
6. KCP emits kit_run_report for ingestion
7. ICP/MCP ingests results and updates ledgers/snapshots

---

7) Kit Failure Semantics

If execution fails, kit must still contain:

- kit_validation_report

- kit_run_report with failure reason + remediation
- logs for executed commands
- partial RESULT/proof artifacts preserved

---

8) Hard Boundaries (kit-level)

- kit must not contain secrets
- external kits must not contain restricted knowledge
- KCP must not invent policies or standards outside kit snapshot
- all outputs must be pointer-linked and schema-valid

Agent Kit Goals

Agent Kit Goals (non-negotiable)

G-01 Self-contained

Meaning

- The kit contains everything the build agent and KCP need to execute without reaching back into AXION or relying on “tribal knowledge.”

Must include

- manifest + entrypoint + pinset
- all required specs/templates/standards/policies for the declared scope
- plan units + acceptance requirements when planning is enabled
- verification policy (what commands to run, what evidence to capture)

Must not require

- hidden files in the repo that aren’t referenced in the manifest
- unrecorded operator instructions
- “go ask the internal agent” to fill missing context

Pass condition

- KCP can validate the kit and start execution using only what’s inside the kit and the declared repo target path.

G-02 Deterministic

Meaning

- The kit produces repeatable decisions and outputs when executed under the same inputs and policy.

Determinism applies to

- plan unit ordering decisions
- verification command list and execution rules
- required outputs and evidence pointer rules
- template/standards versions (pinned)

Allowed nondeterminism

- timestamps, run ids, environment fingerprints (must be isolated)

Pass condition

- two executions of the same kit under the same policy produce:
    - the same unit ordering and requirement checks
    - comparable reports/logs once noise fields are normalized
    - the same “what must be done” outcome

G-03 Enforceable

Meaning

- Rules are expressed in a way that KCP can evaluate mechanically, not as prose expectations.

Must be executable

- required artifacts list (manifest completeness checks)

- schema validations for key artifacts
- 1-target-per-unit enforcement (when plan units exist)
- verification policy checks (required commands must run)
- proof requirements (required proof types and minimum metadata)
- guardrails (forbidden paths, reuse constraints, secrets/PII rules)

Pass condition

- KCP can block progress (BLOCKED/FAILED) when a required rule is violated and can point to the exact remediation.

G-04 Auditable

Meaning

- The kit produces outputs that ICP/MCP can ingest without interpretation.

Must produce

- structured RESULT files per completed unit (with implementation_refs + proof_refs)
- kit_run_report with unit states, attempt chains, verification results, and evidence pointers
- logs for executed verification commands
- proof objects/ledger if enabled

Pointer requirements

- every referenced pointer must dereference (path exists, hashes match if required)
- all outputs reference schema versions and pinset context

Pass condition

- ICP/MCP can ingest the kit outputs and update ledgers/gates without needing manual reconstruction of what happened.

G-05 Least-privilege

Meaning

- The kit contains only what is necessary for the declared targets and the executor type.

Scoping dimensions

- declared targets (platforms/domains/features)
- executor type (internal vs external)
- risk class (prototype vs production vs compliance)

Must enforce

- external kits exclude restricted knowledge and internal-only artifacts
- forbidden path rules are stricter for external execution
- reuse allowlist rules apply where reuse is permitted
- secrets never included; sensitive intake data minimized/redacted as required

Pass condition

- the kit cannot be used as a “data dump” or backdoor into restricted information, and execution is constrained to the declared scope.

1.

Agent Kit Composition

Agent Kit Composition (what it contains)

2.1 Entry artifacts (always)

2.1.1 `KIT_MANIFEST`

Role: authoritative inventory of kit contents; the kit's "truth table."

Must include

- kit identity: `kit_id`, source `run_id`, kit variant (`internal`|`external`)
- pinset reference/hash (or embedded pinset)
- artifact inventory list:
  - `artifact_id` / `artifact_type`
  - `path`
  - `hash` (at minimum sha256)
  - `schema_id` + `schema_version` (where applicable)
  - classification: `required` | `optional`
- integrity section:
  - total file count (optional)
  - bundle hash (if packaged)

Must enable

- deterministic lookup of every required artifact by path
- validation that nothing required is missing or mutated

2.1.2 `KIT_ENTRYPOINT`

Role: ordered execution and reading guide; the kit's "operating instructions."

Must include

- reading order:
  - list of pointers to "read first" artifacts (canonical spec, standards snapshot, templates, plan)
- execution steps:
  - how to start KCP (if shipped)
  - how to execute units (if planning enabled)
  - where to write results/proofs/reports
- references:

    - pointer to plan units directory (if present)
    - pointer to verification policy
    - pointer to guardrail policies (external restrictions, reuse rules)

Must enable

- a build agent to start without searching the kit
- deterministic guidance (no optional ambiguity about “what’s next”)

2.1.3 `PINSET / LOCKFILE`

Role: freezes the run’s versions so results are reproducible.

Must include

- versions for:
    - templates used
    - standards snapshot
    - gate set profile (if kit enforces gates locally)
    - schema versions used for validation
    - verification policy version/toolchain expectations
- registry fingerprints:
    - hashes/versions of relevant registries used to produce the kit

Must enable

- repro: same inputs + same pinset → same kit outputs and comparable execution results

---

2.2 Spec + constraint artifacts (included by profile)

2.2.1 `CANONICAL_SPEC` package

Role: the authoritative “what is being built” spec.

Must include

- canonical spec artifact(s) required by the selected templates
- unknowns/assumptions artifact if applicable
- references to any supporting spec indexes (e.g., SPEC_INDEX if used)

Must enable

- filling remaining template fields and guiding implementation without guesswork

---

2.2.2 `STANDARDS_SNAPSHOT`

Role: the authoritative “rules for this run.”

Must include

- resolved standard IDs + versions
- precedence/conflict decisions
- resolver trace reference (or explicit ordering)
- snapshot hash

Must enable

- KCP to enforce constraints and validate compliance locally (where applicable)

2.2.3 `TEMPLATE_OUTPUTS`

Role: the filled template deliverables produced by ICP.

Must include

- `render_envelopes[]` (each with template_id/version + completeness state)
- template selection result (if produced)
- any completeness report (if separate)

Must enable

- direct consumption by build agent/KCP without needing ICP

2.2.4 Applicability/selection outputs *(optional but recommended)*

Role: audit trail for “why these rules/templates were chosen.”

Should include

- standards applicability output (selection rationale)
- template selection rationale (selection criteria)

Must enable

- post-run audit/debug without re-deriving selection decisions

---

2.3 Planning artifacts (when planning enabled)

2.3.1 `PLAN_UNITS` (1 target per unit)

Role: executable tasks the build agent performs.

Must include

- unit_id
- exactly one target per unit
- acceptance requirements
- required proofs (or pointers to acceptance map)

Must enable

- deterministic execution order and completion tracking

2.3.2 `ACCEPTANCE_MAP`

Role: objective definition of “done.”

Must include

- mapping: unit_id → acceptance criteria → required proof types
- minimum evidence requirements per unit

Must enable

- KCP to enforce DONE criteria mechanically

2.3.3 `BUILD_PLAN / SEQUENCING`

Role: deterministic ordering rules for units.

Must include

- unit ordering (or ordering rules)
- dependency edges if any
- tie-breaker rules (no ambiguous ordering)

Must enable

- KCP to select the “next unit” deterministically

2.4 Execution + verification artifacts (always for serious builds)

2.4.1 `VERIFICATION_POLICY`

Role: defines what must be executed to prove correctness.

Must include

- command list with IDs:
  - required/optional flags
  - expected outputs
  - log capture rules
- retry policy (if any)
- environment/tool requirements (if pinned)

Must enable

- KCP to run verification without inventing commands

2.4.2 `PROOF_TYPE` snapshot *(if not embedded elsewhere)*

Role: defines what proof types exist and what fields they require.

Must include

- proof types allowed for this kit
- required fields per proof type
- acceptable sources (commands/manual attestations)

Must enable

- KCP to normalize proof objects correctly

2.4.3 `EVIDENCE POINTER RULES`

Role: allowed pointer formats and resolver rules.

Must include

- allowed pointer schemes (file/log/commit/hash/artifact)
- resolver validation rules (dereference required, hash match required)

Must enable

- mechanical validation of evidence references in RESULT/proof artifacts

---

2.5 Kit Control Plane (KCP) runtime (when shipped)

2.5.1 KCP executable/CLI

Role: enforcement engine for kit-local execution.

Must include

- ability to validate kit
- run verification policy
- manage units + write reports/results

2.5.2 KCP configs

Role: defines KCP's local enforcement boundaries.

Must include

- allowed commands (command allowlist from verification policy)
- allowed paths and forbidden paths
- external restrictions (if external kit)

2.5.3 Schema validators *(as needed)*

Role: local schema enforcement without contacting ICP.

Must include

- validators for core artifacts: manifest, entrypoint, plan units, results, proof objects

---

2.6 Safety + governance artifacts

2.6.1 `REUSE_ALLOWLIST` *(if reuse is permitted)*

Role: defines what content may be reused verbatim.

Must include

- allowed paths/snippets (or KIDs)
- license/ownership metadata
- excerpt limits and attribution requirements

2.6.2 Plagiarism/IP policy reference (or embedded)

Role: prevents code-donor behavior.

Must include

- restrictions for internal vs external execution
- required reuse logging behavior

2.6.3 Secrets/PII handling rules (especially for external kits)

Role: prevents leakage into logs/kits/results.

Must include

- redaction rules
- retention rules for logs/results
- forbidden content patterns (high level)

2.6.4 Audit/override policy subset *(if KCP must enforce local overrides)*

Role: controls any local override mechanism.

Must include

- never-overridable rules list (kit-local)
- override record requirements (who/why/expiry/scope)
- linkage requirements to kit_run_report

Agent Kit Variants

Agent Kit Variants (by executor type)

3.1 Internal Kit

Purpose: maximize build capability and speed while staying within AXION contracts.

What it may include

- broader reference material:
  - larger knowledge bundle selections
  - internal-only patterns/checklists (where allowed by policy)
- wider tooling surface:
  - more verification tools (profilers, linters, generators) if pinned and policy-approved
  - richer diagnostics utilities
- broader repo access *within bounded rules*:
  - allowed to touch more directories/modules if the plan units require it
- heavier debug configuration:
  - expanded logging settings
  - additional trace/diagnostic outputs

What it must still enforce

- pinset discipline (no unpinned tool drift)
- proof capture per evidence policy
- secrets/PII rules (never in kits/logs/results)
- scope constraints from the run profile and plan units

Failure posture

- may allow richer debug output to accelerate remediation
- still must produce complete audit artifacts (reports/log pointers/results)

3.2 External Kit

Purpose: enable execution by external agents while minimizing leakage and enforcing strict boundaries.

What it must exclude

- restricted knowledge items (internal-only KIDs, proprietary docs)
- unnecessary canonical details beyond what is required to execute targets

- any sensitive intake artifacts not required for execution (or must be redacted/minimized)

What it must include

- stricter guardrails:
  - path allowlists + forbidden path rules
  - “no copy / no donor code” enforcement rules
  - command allowlist (only verified commands)
- explicit reuse controls:
  - `REUSE_ALLOWLIST` behavior defined and enforced
  - reuse logging requirements (if reuse is allowed at all)
- minimal exposure packaging:
  - only the templates/spec sections needed for the declared targets
  - minimal logs/configs that do not disclose internal system details

Execution restrictions

- narrower command set (verification policy is tighter)
- narrower file system access
- stricter evidence pointer rules (no external refs unless allowed)

Failure posture

- fail-fast on any boundary violation
- preserve audit artifacts but redact aggressively per policy

---

Agent Kit Directory Layout (logical)

4.1 Layout purpose

This layout defines where every class of kit artifact lives so:

- KCP can validate and locate artifacts deterministically
- ingestion back into AXION can map outputs reliably
- internal/external variants can be enforced by path policies

4.2 Required directories (logical)

- `kit/manifest/`
  - contains the authoritative inventory and integrity metadata
- `kit/entrypoint/`
  - contains reading order and execution instructions

- `kit/pins/`
  - contains the pinset/lockfile and any pin-resolution metadata
- `kit/spec/`
  - canonical spec package and supporting spec indexes (as included)
- `kit/standards/`
  - standards snapshot + applicability outputs (as included)
- `kit/templates/`
  - render envelopes, template selection result, completeness reports
- `kit/plan/`
  - plan units, acceptance map, build plan/sequencing (if planning enabled)
- `kit/verify/`
  - verification policy, command configs, expected output rules
- `kit/proof/`
  - proof objects and optional kit-local proof ledger (if enabled)
- `kit/kcp/` *(optional)*
  - KCP executable/runtime and configuration
- `kit/policies/`
  - reuse allowlist, IP policy references, secrets/PII handling rules, override subset (if used)

4.3 Runtime output directories (written during execution)

- `kit/logs/`
  - command logs and execution logs (structured, deterministic naming)
- `kit/results/`
  - `RESULT_<UNIT_ID>.json` outputs and any indexing/tracking artifacts
- `kit/reports/`
  - `kit_validation_report`
  - `kit_run_report`
  - `guardrail_report` (if produced)

4.4 Variant enforcement via layout

- external kits must enforce stricter rules primarily by:
  - restricting what appears under `kit/spec`, `kit/templates`, `kit/policies`
  - restricting write/read permissions and allowed roots during execution
- internal kits may include more content, but must still preserve:
  - deterministic structure
  - pinset discipline
  - auditability across `logs/results/reports`

Agent Kit Directory Layout

Agent Kit Directory Layout (logical)

L-00 Purpose

The kit layout is a **contracted filesystem shape** that enables:

- deterministic discovery (KCP can find everything without guessing)
- integrity validation (manifest hashes match file locations)
- clean separation of *inputs shipped in the kit* vs *outputs produced during execution*
- variant control (internal vs external kits differ by what they include, not by changing structure)

L-01 Root

- `kit/` is the kit root.
  All kit pointers are relative to this root unless explicitly stated otherwise.

Rules

- no required artifacts may live outside `kit/`
- KCP must treat unknown top-level directories as optional unless referenced in the manifest

---

L-02 Input (shipped) directories

`kit/manifest/`

Contains

- `KIT_MANIFEST` (authoritative inventory)
- optional manifest supplements (artifact index, hashes map)

Rules

- manifest must be readable before any other directory is trusted
- manifest is the only authoritative source for “what exists” and “what is required”

`kit/entrypoint/`

Contains

- `KIT_ENTRYPOINT` (reading order + execution instructions)
- optional “quickstart” files, if referenced by entrypoint

Rules

- entrypoint must only reference artifacts present in manifest inventory
- entrypoint must not include secrets or external-only references

---

`kit/pins/`

Contains

- `PINSET/LOCKFILE`
- optional registry fingerprints or resolution metadata

Rules

- pinset must be present before any schema validation that depends on pinned schema versions
- pinset is read-only during kit execution

`kit/spec/`

Contains

- canonical spec package (and supporting spec indexes if used)
- unknowns/assumptions artifacts (if present)

Rules

- spec content must be scoped to declared targets
- external kits must apply minimization/redaction policies here

---

`kit/standards/`

Contains

- resolved standards snapshot
- applicability output (if included)
- any constraint pack references (if included)

Rules

- snapshot must be read-only
- external kits must not include restricted internal standards commentary beyond what's needed to enforce constraints

---

`kit/templates/`

Contains

- template selection result (if included)
- render envelopes (filled template outputs)
- completeness report (if separate)

Rules

- render envelopes are the authoritative "template outputs" for this kit
- any citations/knowledge references required by policy must appear in envelopes (or linked metadata)

`kit/plan/`

Contains *(only when planning enabled)*

- plan units (`PLAN_UNIT_*`)
- acceptance map
- build plan / sequencing rules

Rules

- plan units must be enforceable as 1-target-per-unit
- sequencing must be deterministic (or default ordering rule must be specified)

`kit/verify/`

Contains

- verification policy (commands, required/optional flags, expected outputs, log rules)
- configs needed to run those commands (only if required and safe)

Rules

- command allowlist is derived from verification policy
- external kits must not ship unsafe commands or unrestricted shell behaviors

---

`kit/proof/`

Contains

- proof type snapshot (if shipped separately)
- evidence pointer rules (if shipped separately)
- any proof schema validators (if included)

Rules

- this directory defines “what counts as proof” for kit-local execution
- read-only during execution except for optional kit-local proof ledger if policy allows it

---

`kit/kcp/` ***(optional)***

Contains

- KCP executable/CLI
- KCP configuration (allowed paths, forbidden paths, external restrictions)
- local schema validators (if shipped)

Rules

- if present, entrypoint must specify how to run it
- KCP config must be treated as policy and be immutable during execution

---

`kit/policies/`

Contains

- reuse allowlist (if any)
- plagiarism/IP rules reference or embedded policy
- secrets/PII handling rules
- audit/override subset (if applicable)

Rules

- policy files are read-only
- external kits must include stricter policy variants

L-03 Output (runtime) directories

These are written during execution. They must be cleanly separated from shipped inputs.

`kit/logs/` *(runtime output)*

Contains

- verification command logs
- execution logs as needed

Rules

- deterministic naming by command_id and attempt_id
- redaction rules must apply (no secrets/PII)

`kit/results/` *(RESULT files)*

Contains

- `RESULT_<UNIT_ID>.json`
- optional result index or unit status files

Rules

- results must be schema-valid
- results must include resolvable pointers to implementation/proofs
- results must not overwrite prior attempts (attempt-scoped if needed)

`kit/reports/` *(run reports)*

Contains

- `kit_validation_report`
- `kit_run_report`
- `guardrail_report` (if produced)

Rules

- reports must be structured and pointer-linked
- reports must be produced even on failure (partial but valid)

L-04 Variant constraints (internal vs external)

Layout stays the same. Variant differences are enforced by:

- what is included under `kit/spec`, `kit/templates`, `kit/policies`, `kit/kcp`
- stricter path rules and command rules for external kits
- minimization/redaction for external kits in shipped content and runtime outputs

---

L-05 Layout invariants (must always be true)

- manifest and entrypoint exist and are schema-valid
- manifest inventory paths map to real files
- all shipped inputs are immutable during execution
- all runtime outputs are written only under `kit/logs`, `kit/results`, `kit/reports` (and optional proof outputs if allowed)
- no secrets included anywhere; redaction applies to logs/reports/results

Kit Contracts

Kit Contracts (schemas you must lock)

5.1 `KIT_MANIFEST` schema

Purpose: make kit contents machine-verifiable and tamper-detectable.

Must define

- **identity**
  - `kit_id`
  - `run_id`
  - `variant` (`internal` | `external`)
- **schema control**
  - `schema_version` (manifest schema version)
  - referenced schema versions used by included artifacts (by schema_id/version)
- **pinning**
  - `pinset_ref` and/or `pinset_hash`
- **artifact inventory** *(list)*
  - `artifact_id`
  - `artifact_type`
  - `path`
  - `hash` (algorithm + value)
  - `schema_id` + `schema_version` (if schema-governed)
  - `required` flag (`required` | `optional`)

    - `access_class` (`public` | `internal` | `restricted`) *(especially for external kits)*

- **integrity**
    - optional `bundle_id` + `bundle_hash` (if exported)
    - optional total counts (files/artifacts)

Must enable

- deterministic “required artifacts present” validation
- hash verification of included artifacts
- variant enforcement (internal/external constraints)

5.2 `KIT_ENTRYPOINT` schema

Purpose: remove ambiguity about what to read and what to do.

Must define

- **reading order**
    - ordered list of artifact pointers (paths or artifact IDs)
- **execution steps**
    - ordered step list with:
        - step id
        - action type (read/execute/verify/write-result)
        - pointer(s) to referenced artifacts/policies
- **unit execution rules** *(if planning included)*
    - 1-target requirement

    - ordering source (build plan vs fallback)
    - completion requirements pointer (acceptance map)
- **verification invocation**
    - pointer to `VERIFICATION_POLICY`
    - when verification must run (per unit / per batch / end-only)
- **result submission instructions**
    - RESULT file schema reference
    - where RESULT files must be written
    - how to link proofs/logs via pointers

Must enable

- KCP to execute the kit without guessing workflow
- deterministic sequencing and validation

5.3 `PLAN_UNIT` schema (kit-local)

Purpose: define atomic build work that can be enforced and proven.

Must define

- **identity**
    - `unit_id`
- **targets**
    - `targets` array with exactly **1** target object:
        - target category/type

        - target id/reference

- **acceptance requirements**
    - acceptance criteria list (or pointer to acceptance map)
- **required proofs**
    - proof type list and minimum metadata expectations
- **output expectations**
    - what artifacts/changes must exist (high-level)
    - required result fields (implementation refs, proof refs)
- **pointers**
    - pointers to relevant spec sections/templates that guide execution

Must enable

- 1-target enforcement
- objective DONE criteria
- deterministic “next unit” ordering when paired with sequencing rules

---

5.4 `VERIFICATION_POLICY` schema

Purpose: define what must be executed to prove correctness and how to capture evidence.

Must define

- **command list**
    - command id
    - command string or runnable reference

    - `required` vs `optional`
    - expected outputs (file paths or artifact pointers)
    - log capture rules (path pattern, stdout/stderr requirements)
- **environment constraints**
    - required tool versions (or references to pins)
    - CI/local allowances (what differs, if anything)
- **failure rules**
    - fail-fast vs continue-on-fail (policy-defined)
    - retry rules (if allowed): max retries, conditions, backoff (if defined)

Must enable

- KCP to run verification deterministically and capture comparable evidence

---

5.5 `RESULT` schema

Purpose: make completion evidence ingestible without interpretation.

Must define

- **identity**
    - `unit_id`
- **implementation evidence**
    - `implementation_refs[]` (non-empty for DONE)
- **proof evidence**
    - `proof_refs[]` (non-empty for DONE)

- **timestamps**
    - created/submitted timestamps (allowed nondeterminism, but required)
- **notes** (optional)
    - free text notes (must not be required for correctness)

Must enable

- ICP/MCP ingestion without “reading between the lines”
- traceable linkage from unit → implementation → proof via resolvable pointers

Kit Execution Contract

Kit Execution Contract (what must happen)

E-00 Purpose

Define the required end-to-end behavior from “kit delivered” to “results ingested,” so execution is deterministic, enforceable, and auditable.

E-01 KCP validates kit integrity

Must happen first

- KCP reads `KIT_MANIFEST`, `KIT_ENTRYPOINT`, and `PINSET/LOCKFILE`.
- KCP verifies required artifacts exist and schemas validate (as required by the kit).
- KCP verifies pinset is parseable and matches manifest expectations.

Required output

- `kit_validation_report` (PASS/FAIL with evidence pointers)

Stop rule

- if validation FAILS, execution stops and `kit_run_report` must still be produced in failed form.

E-02 KCP selects next unit deterministically

Condition

- only applies when planning is enabled and plan units exist.

Must do

- load `PLAN_UNITS` and sequencing rules (build plan / ordering rules).
- enforce 1-target-per-unit.
- determine the next eligible unit using deterministic ordering.

Required output

- update `unit_status_index` (unit list + states + attempt chain)
- optional `next_unit_recommendation` (must be derivable from deterministic rules)

Stop rule

- multi-target unit or invalid unit schema triggers fail-fast per kit policy.

E-03 Build agent implements unit target

Must do

- implement only what the active `PLAN_UNIT` target requires.
- keep changes within allowed scope and allowed paths.
- follow standards/constraints shipped in the kit.

Required intermediate state

- unit transitions to `IN_PROGRESS`.
- any partial outputs must be preserved by attempt id.

Stop rule

- scope breach or non-overridable guardrail violation triggers FAIL.

E-04 KCP runs verification per policy

Must do

- run verification commands defined in `VERIFICATION_POLICY` at the required time:
  - per unit, per batch, or end-only (as specified)
- capture logs (stdout/stderr), exit codes, timestamps
- write logs to deterministic paths

Required output

- `verification_run_result`
- `logs/<command_id>.log` (and/or pointers)

Stop rule

- required command failures follow policy (fail-fast or block for remediation).

E-05 KCP produces RESULT + proof objects

Must do

- create `RESULT_<UNIT_ID>.json` for the unit when it meets acceptance requirements.
- attach:
  - `implementation_refs[]` (non-empty)
  - `proof_refs[]` (non-empty, resolvable pointers)
- generate proof objects (if enabled/required) consistent with proof type snapshot.

Required output

- `RESULT_<UNIT_ID>.json`
- proof objects and/or kit-local proof ledger (if enabled)

Completion rule

- unit cannot be marked `DONE` without valid RESULT fields and required proof refs.

E-06 KCP emits `kit_run_report` for ingestion

Must do

- produce a structured run report regardless of success/failure.
- include:
  - unit states + attempt chain
  - verification results + log pointers
  - RESULT file pointers
  - proof pointers (if any)
  - guardrail violations (if any)
  - remediation block on failure

Required output

- `kit_run_report`

E-07 ICP/MCP ingests results and updates ledgers/snapshots

Must do

- ICP/MCP reads `kit_run_report` and all referenced artifacts.
- validates:
  - RESULT schemas

    - proof schemas
    - evidence pointers resolve
- updates:
    - proof ledger (append/revision)
    - acceptance/completion status for units/targets
    - run/stage reports as applicable
    - state snapshot / continuity pointers as applicable

Required outcome

- ingestion produces a deterministic “completion state” update that can be audited and reproduced.

Stop rule

- ingestion must fail if:
    - required RESULT/proof artifacts are missing or invalid
    - evidence pointers do not resolve
    - required proofs are absent for declared completion

Kit Failure Semantics

Kit Failure Semantics

F-00 Purpose

A failed kit execution must still be **auditable and restartable**. Failure cannot leave the system guessing what happened or require rerunning blindly.

F-01 Minimum artifacts that must exist on failure

If execution fails at any point, the kit outputs must still include:

F-01.1 `kit_validation_report`

- must exist whether validation passed or failed
- if validation failed, it must:
  - identify missing/invalid artifacts
  - include evidence pointers to the exact files/pointers that failed
  - state that execution did not proceed beyond validation

F-01.2 `kit_run_report` with failure reason + remediation

Must include:

- final kit-run state (`BLOCKED` or `FAILED`)
- failure location:
  - phase (`KIT_VALIDATION`, `EXECUTING`, `VERIFYING`, `GUARDRAILS`)
  - unit_id (if unit-related)
- failure reason codes (not only free text)
- remediation steps (actionable and ordered)
- evidence pointers to:
  - failing artifact(s)
  - failing logs
  - guardrail report section (if applicable)
- attempt chain summary (if any unit attempts were started)

F-01.3 Logs for executed commands

If any verification commands were executed:

- logs must be present and referenced by pointers in the run report
- logs must include:

    - command id
    - exit code
    - stdout/stderr capture
    - attempt id if rerun occurred

F-01.4 Partial RESULT/proof artifacts preserved

If any RESULT files or proof objects were created:

- they must not be overwritten or deleted
- they must remain referenceable via pointers
- if partial/incomplete, they must be clearly marked as such (status field or absence of DONE markers)

F-02 Failure classification (deterministic)

The kit run must classify failure into at least:

- **BLOCKED** (recoverable under same kit/policy)
    - missing proof refs that can be added
    - verification failure with retry permitted by policy
    - incomplete implementation within scope
- **FAILED** (non-recoverable under current kit/policy)
    - invalid/missing kit artifacts
    - schema invalid artifacts
    - non-overridable guardrail violation (forbidden paths, secret leakage, restricted content access)
    - pointer resolution failure for required evidence

Classification must appear in `kit_run_report` as reason codes.

F-03 Fail-fast conditions

Execution must stop immediately (and still write the failure artifacts) if:

- required kit artifacts are missing
- core schemas fail validation (manifest/entrypoint/pinset/plan units)
- a plan unit violates 1-target rule
- a non-overridable guardrail violation occurs
- required verification commands fail and policy is fail-fast

---

F-04 Attempt preservation rules

If a failure happens mid-unit:

- the current attempt must be recorded
- reruns create new attempt records (no overwrite)
- previous logs and partial outputs remain immutable and referenced

---

F-05 “Failure is well-formed” criteria

A kit failure is considered acceptable only if:

- failure phase and unit_id (if applicable) are identified
- remediation steps are present
- evidence pointers resolve
- partial outputs are preserved and referenced

Hard Boundaries

Hard Boundaries (kit-level)

B-00 Purpose

Kit-level hard boundaries prevent leakage, policy drift, and unverifiable outputs. These are **non-negotiable** constraints enforced by kit composition, KCP guardrails, and schema validation.

B-01 Kit must not contain secrets

Meaning

- No credentials, API keys, private keys, tokens, or secret config values may be included anywhere in the kit.

Applies to

- shipped inputs (`kit/spec`, `kit/templates`, `kit/policies`, etc.)
- runtime outputs (`kit/logs`, `kit/results`, `kit/reports`)

Required enforcement

- pre-packaging scan at ICP time (fail kit packaging if secrets are detected)
- runtime redaction and detection rules at KCP time (fail-fast if secrets appear in logs/results)

Failure rule

- secret presence is a non-overridable violation for external kits; internal kits may only proceed if policy explicitly allows a controlled override and logs it (default: no).

B-02 External kits must not contain restricted knowledge

Meaning

- External kits cannot ship internal-only knowledge items, proprietary docs, or restricted reference content.

Restricted content examples

- internal playbooks, proprietary architectures, internal code examples
- knowledge items marked `restricted_internal_only`
- content with licensing that disallows redistribution

Required enforcement

- kit assembly must filter by:
  - executor type = external
  - knowledge item `executor_access` and `use_policy`
- manifest must label access class for included knowledge artifacts

Failure rule

- inclusion of restricted knowledge in an external kit is a non-overridable failure.

B-03 KCP must not invent policies or standards outside kit snapshot

Meaning

- KCP is an executor. It cannot change what rules apply or introduce new enforcement logic beyond what the kit declares.

Allowed

- applying the shipped standards snapshot, verification policy, evidence pointer rules
- enforcing shipped guardrails as written

Not allowed

- selecting new standards/templates not present in the kit
- modifying proof requirements
- creating new predicate operators or relaxing existing rules
- executing commands not listed in verification policy

Required enforcement

- command allowlist derived strictly from `VERIFICATION_POLICY`
- operator registry/policy registry snapshots treated as read-only (if shipped)
- all enforcement decisions must reference the policy artifact that authorized them

---

B-04 All outputs must be pointer-linked and schema-valid

Meaning

- Anything KCP produces must be machine-ingestible by ICP/MCP without interpretation.

Applies to

- `RESULT_<UNIT_ID>.json`
- `kit_run_report`, `kit_validation_report`, `guardrail_report`
- proof objects / kit-local proof ledger (if used)
- verification run results (if separate artifact)

Required enforcement

- schema validation for every produced artifact before it is considered “written”
- evidence pointers must dereference:
    - files must exist at paths
    - hashes must match when required
- incomplete/partial artifacts must be clearly marked and must not claim completion

Failure rule

- inability to validate schema or resolve required pointers is a fail-fast condition (BLOCKED if fixable; FAILED if not).